



Java Useful Classes

Nazanin Foroutan
AP Spring 2026 – Dr. Mojtaba Vahidi Asl



Don't Reinvent the Wheel!

- Java provides **built-in classes** with ready-to-use methods.
- These classes solve common problems: Math, Text manipulation, Data conversion, Arrays.
- **The Golden Rule:** If you are writing a loop to do something simple (like sorting), check if a Utility class already does it.
- Key classes to learn today are as following:

Math

String

StringBuilder

Arrays

Scanner

LocalDate

Wrapper classes



Reading from the Keyboard

- `nextInt()`, `nextDouble()` → for numbers
- `nextLine()` → for full sentences
- `next()` → for a single word

```
1 import java.util.Scanner;
2
3 Scanner scanner = new Scanner(System.in);
4 System.out.print("Enter your age: ");
5 int age = scanner.nextInt();
6
7 System.out.print("Enter your name: ");
8 String name = scanner.nextLine(); // Note: consumes newline character
9
10 scanner.close(); // Good practice
```



Math Class – Static Methods

- `Math.PI (3.14159...)`

Method	What it does	Example
<code>Math.max(a, b)</code>	Returns larger number	<code>Math.max(10, 20) → 20</code>
<code>Math.min(a, b)</code>	Returns smaller number	<code>Math.min(10, 20) → 10</code>
<code>Math.pow(a, b)</code>	a raised to power b	<code>Math.pow(2, 3) → 8.0</code>
<code>Math.sqrt(a)</code>	Square root	<code>Math.sqrt(25) → 5.0</code>
<code>Math.abs(a)</code>	Absolute value	<code>Math.abs(-5) → 5</code>

Math.random() & Range Formulas

- (int) = casting (removes decimals)
- $*(\text{max} - \text{min} + 1) + \text{min}$

```
1 // Returns double: 0.0 <= number < 1.0
2 double rand = Math.random(); // e.g., 0.734
3
4 // Formula for integer range: (min to max)
5 int diceRoll = (int)(Math.random() * 6) + 1; // 1 to 6
6 int randomGrade = (int)(Math.random() * 101); // 0 to 100
```



The Immutable Heart of Java

- **The String Pool Concept (Important!):**
- Java stores literal strings ("Hi") in a special memory area called the **String Pool**.
- If you create "Hi" again, Java reuses the same object.
- **Immutability:** Once a String is created, it cannot be changed.
 - `s1.toUpperCase()` does NOT change `s1` - it returns a *new* String.



```
1 String s1 = "Hello";  
   // Goes to String Pool  
2 String s2 = new String("Hello");  
   // Creates new object (avoid this)
```



Manipulating Text (Creates New Strings)

```
1 String text = " Java Programming ";
2
3 text = text.trim();           // "Java Programming"
4 boolean isEqual = text.equals("Java"); // false
5 String lower = text.toLowerCase(); // "java programming"
6 char letter = text.charAt(0);    // 'J'
7
8 // Advanced
9 String replaced = text.replaceAll("Java", "Python");
10 String[] words = text.split(" "); // ["Java", "Programming"]
11
12 String formatted = String.format("Score: %d", 100);
```

- **Reminder:** These methods return *new* Strings; the original is unchanged.



StringBuilder Class (Mutable)

- When you are appending text inside a loop or making many changes.
- **Performance**
Note: StringBuilder is much faster than String for loops (> 10x faster).

```
1  StringBuilder sb = new StringBuilder("Hello");
2
3  sb.append(" World");           // "Hello World"
4  sb.insert(5, ",");            // "Hello, World"
5  sb.delete(5, 6);              // Removes the comma → "Hello World"
6  sb.reverse();                 // "dlroW olleH"
7
8  String finalText = sb.toString(); // Convert to normal String
```





Mutable vs Immutable (Comparison Table)

Analogy: String = a printed book (can't edit). StringBuilder = a Word document (edit freely).

Feature	String	StringBuilder
Mutability	Immutable (cannot change)	Mutable (can change)
Memory	String Pool	Heap memory only
Performance	Slow with many changes	Fast with many changes
Thread-safe	Yes	No (not needed for students)
Best for	Storing fixed text (names, IDs)	Building text dynamically (loops, reports)

Wrapper Classes

- Converting Primitives to Objects
- **Why?** Collections (ArrayList, HashMap) only work with Objects, not primitives.
- **Auto-boxing (Java does this automatically)**

Primitive	Wrapper Class
int	Integer
double	Double
char	Character
boolean	Boolean

```
1 Integer num = 42;           // Auto-boxing (int → Integer)
2 int value = num;            // Auto-unboxing (Integer → int)
```



Integer Class

```
1 Integer i = 100;
2
3 int primitive = i.intValue();      // 100 (rarely needed)
4 double asDouble = i.doubleValue(); // 100.0
5
6 // Static methods (use with class name)
7 int max = Integer.MAX_VALUE;      // 2147483647
8 int min = Integer.MIN_VALUE;      // -2147483648
9
10 int parsed = Integer.parseInt("123");
   // Convert String to int
11 int comparison = Integer.compare(10, 20); // -1 (10 < 20)
```



Character Class

- **Use Case:** Validating user input (e.g., checking if a password contains a digit).

```
1 char ch = 'A';  
2  
3 boolean isDigit = Character.isDigit('5');    // true  
4 boolean isLetter = Character.isLetter('Z');   // true  
5 boolean isLower = Character.isLowerCase('a'); // true  
6 boolean isUpper = Character.isUpperCase('A'); // true  
7  
8 char toLower = Character.toLowerCase('B');    // 'b'  
9 char toUpper = Character.toUpperCase('c');     // 'C'
```



Arrays Class

- **All methods are static**
- **Call with** `Arrays.methodName()`

Method	What it does	Example
<code>sort(array)</code>	Sorts in ascending order	<code>Arrays.sort(nums);</code>
<code>binarySearch(array, key)</code>	Finds index (array must be sorted)	<code>int index = Arrays.search(nums, 5);</code>
<code>fill(array, value)</code>	Fills all elements with a value	<code>Arrays.fill(scores, 0);</code>
<code>equals(arr1, arr2)</code>	Checks if two arrays are identical	<code>Arrays.equals(a, b) → boolean</code>
<code>copyOf(original, newLength)</code>	Copies array	<code>int[] copy = Arrays.copyOf(nums, 10);</code>
<code>toString(array)</code>	Prints array nicely	<code>System.out.println(Arrays.toString(nums));</code>

Example for Array classes

- You might like:

```
1 int[] scores = {45, 12, 89, 33};  
2 Arrays.sort(scores);  
3 System.out.println(Arrays.toString(scores));  
  // [12, 33, 45, 89]
```



LocalDate & LocalTime Classes

Why LocalDate instead of old Date?

- Old `Date` class is confusing (months start at 0, years are weird).
- `java.time` package (Java 8+) is clean, intuitive, and thread-safe.
- Import Statement:

```
1 import java.time.LocalDate;  
2 import java.time.LocalDateTime;  
3 import java.time.format.DateTimeFormatter;
```



Creating Dates

```
1 // Current date
2 LocalDate today = LocalDate.now();           // 2026-04-20
3
4 // Specific date (year, month, day)
5 LocalDate birthday = LocalDate.of(2005, 5, 15); // May 15, 2005
6
7 // From String (ISO format: YYYY-MM-DD)
8 LocalDate parsedDate = LocalDate.parse("2024-12-25");
```



Common Methods

Method	What it does	Example
<code>getYear()</code>	Extracts year	<code>today.getYear()</code> → 2026
<code>getMonthValue()</code>	Month as number (1-12)	<code>today.getMonthValue()</code> → 4
<code>getMonth()</code>	Month as enum (APRIL)	<code>today.getMonth()</code> → APRIL
<code>getDayOfMonth()</code>	Day of month	<code>today.getDayOfMonth()</code> → 20
<code>getDayOfWeek()</code>	Day name	<code>today.getDayOfWeek()</code> → MONDAY
<code>plusDays(n)</code>	Add n days	<code>today.plusDays(10)</code>
<code>minusMonths(n)</code>	Subtract n months	<code>today.minusMonths(1)</code>
<code>isBefore(other)</code>	Checks if earlier	<code>birthday.isBefore(today)</code>
<code>isAfter(other)</code>	Checks if later	<code>birthday.isAfter(today)</code>

Example: Age Calculator

```
1 LocalDate birthDate = LocalDate.of(2006, 8, 22);
2 LocalDate currentDate = LocalDate.now();
3
4 int age = currentDate.getYear() - birthDate.getYear();
5
6 // Check if birthday already passed this year
7 if (currentDate.getDayOfYear() < birthDate.getDayOfYear()) {
8     age--; // Birthday hasn't happened yet
9 }
10 System.out.println("Age: " + age);
```



Example: Formatting Dates for Display

```
1 LocalDate event = LocalDate.of(2026, 12, 25);
2
3 // Built-in formatters
4 String formatted1 = event.format(DateTimeFormatter.ISO_LOCAL_DATE);
5 // 2026-12-25
6
7 // Custom format
8 DateTimeFormatter custom = DateTimeFormatter.ofPattern("MMMM d, yyyy");
9 String formatted2 = event.format(custom); // "December 25, 2026"
10
11 // Short format
12 DateTimeFormatter shortFormat = DateTimeFormatter.ofPattern("MM/dd/yy");
13 String formatted3 = event.format(shortFormat); // "12/25/26"
```





Thanks!

